Tree

A few points:

So far, we have said that the tree is a non linear data structure. A tree is an acyclic, connected graph. A tree of n nodes has n -1 branches or edges. If we add an edge, a cycle is formed. If we remove an edge, the connectivity between every pair of nodes is lost.

A general tree is defined as

a) Either empty or
b) Has a root followed by number of subtrees.

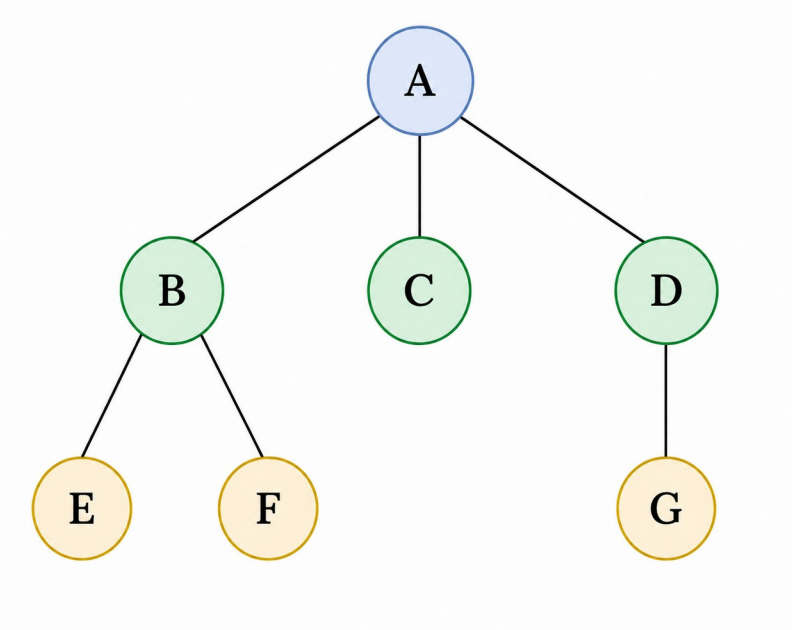
Some of the terms we use with the tree are:

Root, nodes, internal nodes, leaves, branches, path, ancestor, descendant

Parent, child, grand parent, grandchild, siblings, uncle

Height, depth, level.

Degree of a node.



A possible ADT for a general tree :

- Create a tree
- Destroy the tree
- Check empty
- Check full
- Add a node
- Remove a node
- Traverse a tree
- Find root
- Find children
- Check whether leaf
- Find height of the tree
- Find the path from the root to a given node

Binary Tree:

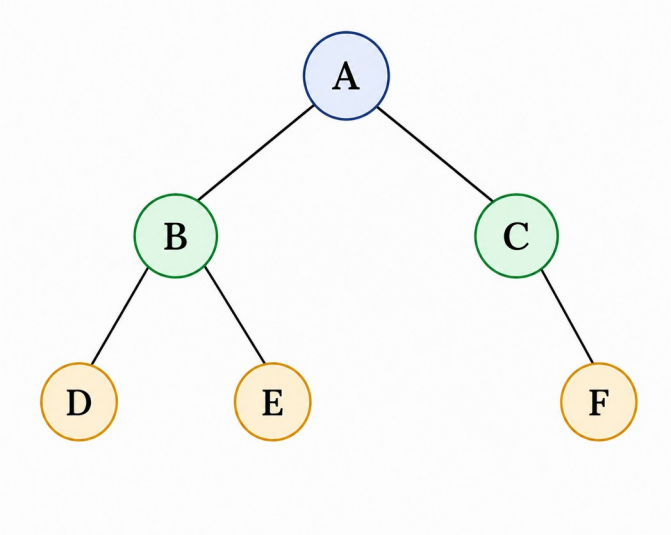
A **binary tree** is a tree in which every node has **at most two children**. The two children are distinguished as the **left child** and **right child**.

A binary tree is either:

- An **empty tree**, or
- A root node together with two disjoint binary trees called its:
    - **left subtree**
    - **right subtree**

Please observe the recursive nature of this definition. As the data structure is inherently recursive, writing recursive functions to support the interface is easy.

An example :



- A is the root.
- B is the left child of A.
- C is the right child of A.
- D and E are children of B.
- F is the right child of C.
- D, E, and F are leaves.

Binary Tree : Abstract Data Type

These are possible operations on a binary tree.

Create()
Create an empty tree
Destroy()
Remove the tree
root()
Return the root of the tree.
parent(p)
Return the parent of node p.
left(p)
Return the left child of p.
right(p)

Return the right child of p.
isLeaf(p)
Test whether p is a leaf.
component(p)
Return the component stored at p.
addRoot(x)
Create a root containing x.
addLeft(p, x)
Add x as the left child of p.
addRight(p, x)
Add x as the right child of p.
remove(p)
Remove node p and its subtree.
size()
Return the number of nodes.
Is_empty()
Test whether the tree is empty.
Is_full()
Check whether a component can be added
height()
Return the height of the tree.

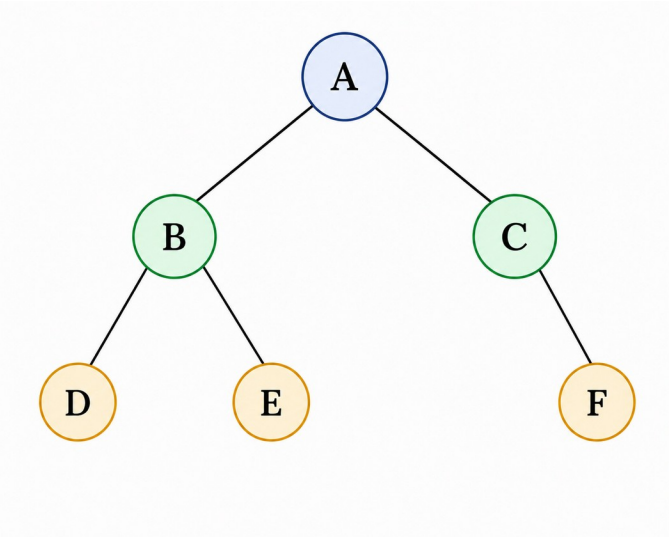
We may implement a few of these interfaces in the next sections.

Traversals:

1. Inorder traversal of a binary tree:

We can visit all the components in an array or a list from one end to another. It is not possible to do so in case of a tree. One of the ways of visiting all the components in a binary tree is to visit the left subtree somehow and then the root and then the right subtree somehow. If the tree is

empty, there is nothing to visit. As the left and the right subtrees are trees, we could apply this logic recursively.



Before visiting A, we visit left sub tree of A – that is the subtree B.

Before visiting B, we visit left sub tree of B – that is the subtree D.

Left sub tree of D is empty. So **visit D**.

Right sub tree of D is empty. So the sub tree D has been visited.

Left sub tree of B has been visited. So visit B.

Now visit the right sub tree of B.

Next we **visit E.**

Now, the left sub tree of A is visited. So **visit A** and then its right subtree.

This results in **visit C** and then **visit F.**

So the traversal of this tree will result in :

D B E A C F

So, in inorder traversal of a binary tree,

If the tree is not empty

We visit the left subtree in inorder

Then the root

Then visit the right subtree in inorder

2. **preorder traversal of a binary tree:**

   In this traversal,

   If the tree is not empty,

   We visit Then the root

   Then left subtree in preorder

   Then visit the right subtree in preorder

   For the tree above, the preorder traversal will result in the following listing.

   A B D E C F

3. **postorder traversal of a binary tree:**

   In this traversal,

   If the tree is not empty,

   We visit left subtree in postorder

   Then visit the right subtree in postorder

   Then the root

   For the tree above, the postorder traversal will result in the following listing.

   D E B F C A

Implementation of Binary Search Tree:

To start with, we will make a simple example of binary trees. We will use a special binary tree called binary search tree.

In a binary search tree, every component has a key. The components are arranged based on the key.

If the tree is empty, the component is put at the root.

Otherwise, if the key in the component at the root is higher than the key in the new component, we will insert in the left subtree.

Otherwise, we will in the right subtree.

You will observe that the insertion always happens as a leaf.

One of the implementations will be as follows.

The tree has a root. This is like a list having a head.

The root points to a node structure.

The node structure has a component – called key – in this example.

Apart from that, it has two pointers – which point to the left and the right sub trees.

```
struct tree
{
        node_t *root_;
};
typedef struct tree tree_t;

struct node
{
        int key_;
        struct node *left_;
        struct node *right_;
};
typedef struct node node_t;
```

This is the simplest interface to start experimenting.

void init(tree_t *ptr_tree);

- Ground the root.

void deinit(tree_t *ptr_tree);

- Walk through the tree
- Remove the left subtree
- Remove the right subtree
- Remove the root

void disp(tree_t *ptr_tree); // This is inorder traversal of the tree

- If the tree is not empty
    - Display the left subtree
    - Display the root
    - Display the right subtree

void insert(tree_t *ptr_tree, int key);

- If the tree is empty make this the root
- Otherwise compare the keys
- Decide to insert in the left subtree or the right subtree

We mentioned that the tree is inherently a recursive data structure.

But the tree structure by itself is not recursive – the node structure which is self-referential is recursive. So, the functions in the tree call their corresponding recursive counter part in nodes.

Let us have a look at them.

Traversal:

Observe that the tree function disp_inorder calls disp_recursive_inorder of the node.

The disp_recursive_inorder_function of the node calls itself on its left subtree, displays the root and Calls itself on its right subtree.

```
void disp_inorder(tree_t *ptr_tree)
{
	printf("Tree inorder: ");
	disp_recursive_inorder(ptr_tree->root_);
	printf("\n");
}

static void disp_recursive_inorder(node_t* temp)
{
	if(temp)
	{
		disp_recursive_inorder(temp->left_);
		printf("%d ", temp->key_);
		disp_recursive_inorder(temp->right_);
	}
}
```

The other two traversals are similar.

Check the files in the folder : inorder-preorder-postorder.

Insert:

```
void insert(tree_t *ptr_tree, int key)
{
        node_t *temp = (node_t*)malloc(sizeof(node_t));
        temp->key_ = key;
        temp->left_ = temp->right_ = NULL;
        ptr_tree->root_ = insert_recursive(ptr_tree->root_, temp);
}
```

The above tree routine creates a node. Assumes that it succeeds in memory allocation. Populates the node. Calls the insert_recursive function of the node to return the root.

This function is very interesting. If the tree is empty, then this new node becomes the root. Otherwise based on the comparison of the key, we try to insert in the left subtree or the right subtree. If the insertion is in the left subtree, then left of the root is the root for that subtree. Similarly, if the insertion is in the right subtree, the right of the root is the root for that subtree. So, the return value of the function is assigned to the appropriate root.

You may want to draw and check how this works for a tree of at least 3 nodes.

```
static node_t* insert_recursive(node_t* root, node_t* temp)
{
	if(root == NULL)
	{
		root = temp;
	}
	else if(root->key_ > temp->key_)
	{
		root->left_ = insert_recursive(root->left_, temp);
	}
	else
	{
		root->right_ = insert_recursive(root->right_, temp);
	}
	return root;
}
```

Deinitialize:

Like other functions, deinit of tree calls the deinit_recursive of node.

```
void deinit(tree_t *ptr_tree)
{
	deinit_recursive(ptr_tree->root_);
	ptr_tree->root_ = NULL;
}
```

This recursive function on the node clears the left subtree recursively, then the right subtree recursively and then the root.

Please node that the order matters.

```
static void deinit_recursive(node_t* root)
{
	if(root)
	{
		deinit_recursive(root->left_);
		deinit_recursive(root->right_);
		free(root);
	}
}
```

Adding a few more functions in the interface of the tree:

The following functions indicate how we formulate recursive routines on binary search trees.

Most of these functions use the simplest model of a binary tree :

- Empty
- Root followed by left and right subtrees

a) Count the number of nodes

If the root is NULL, return 0

Else return 1 + number of nodes in the left subtree + number of nodes in the right subtree

b) Count the number of leaves

If the root is NULL, return 0

Else if the left subtree and the right subtree of the root are empty, return 1

Else return number of leaves in the left subtree and the number of leaves in the right subtree.

c) Check whether the two trees say T1 and T2 are same

If T1 and T2 are empty, then return true

Else if T1 or T2 is empty, then return false

Else return key of root of T1 = key of root of T2 and left subtree of T1 = left subtree of T2 and right subtree of T1 = right subtree of T2

Please check the folder copy_and_compare for the code.

```
// node
int are_same_recursive(node_t* root1, node_t* root2)
{
        if(root1 == NULL && root2 == NULL)
                return 1;
        else if(root1 == NULL || root2 == NULL)
                return 0;
        else
                return root1->key_ == root2->key_ &&
                        are_same_recursive(root1->left_, root2->left_) &&
                        are_same_recursive(root1->right_, root2->right_);
}
// tree
```

```
int are_same(tree_t* ptr_tree1, tree_t* ptr_tree2)
{
        return are_same_recursive(ptr_tree1->root_, ptr_tree2->root_);
}
```

d) Copy one tree to another (say T1 to T2)

If T1 is empty, make T2 empty

Else

Make a node

Copy the key of root of T1 to the new node

Copy the left subtree of T1 to left subtree of the new node created

Copy the right subtree of T1 to right subtree of the new node created

Return the new node

Please check the folder copy_and_compare for the code.

```
// node
node_t* copy_recursive(node_t* ptr_src)
{
    node_t *temp;
    if(ptr_src != NULL)
    {
        temp = (node_t*)malloc(sizeof(node_t));
        temp->key_ = ptr_src->key_;
        temp->left_ = copy_recursive(ptr_src->left_);
        temp->right_ = copy_recursive(ptr_src->right_);
        return temp;
    }
    else
    {
        return NULL;
    }
}
```

```
// tree
void copy(tree_t* ptr_src, tree_t* ptr_dst)
{
    ptr_dst->root_ = copy_recursive(ptr_src->root_);
}
```

Observe the code is somewhat similar to that of insert.

You may want to convince yourself why the copy function is void, but copy_recursive returns a pointer to a node.

Search for a key in a binary search tree:

A binary search tree is organized based on a predicate applied at the root. We use this predicate while inserting as well as searching.

While inserting if the root is empty, the node with the key is made the root.

Otherwise, we compare the key with the key at the root. If the key is less than the key at the root, we insert in the left subtree otherwise in the right subtree.

There are two aspects to searching in a tree.

- How to search?
- What to return?

If the tree is empty, the key is not found.

If the key matches the key at the root, the key is found at the root.

Otherwise if key is less than the key at the root, we search the left subtree recursively

Else search the right subtree recursively.

What should the search function return?

Is it meaningful if the search function displays a message? Absolutely NO. The client cannot access the information found. A function should output something on the screen only if it is for displaying.

Should be fine if the function returns true if the key is found otherwise false? Not really. The client may want to get the component found with respect to the key.

The right way is to have something called an iterator which represents the concept of position. But we are not discussing this in this course. So, we will return a pointer to a node which is not the best.

```
node_t* search_recursive(node_t* root, int key)

{

    if(root == NULL)

            return NULL;

    else if(key == root->key_)

            return root;

    else if(key < root->key_)

            return search_recursive(root->left_, key);

    else

            return search_recursive(root->right_, key);

    }
```

```
node_t* search(tree_t *ptr_tree, int key)
{
    return search_recursive(ptr_tree->root_, key);
}
```

The search function on the tree is non-recursive. It in turn calls the recursive function on the node.

Check the code in the folder search_delete_key

Delete a key :

This is the most complicated function on trees we are encountering so far.

Delete key on the tree is delegated to delete key recursive function on node.

```
void delete_key(tree_t *ptr_tree, int key)
{
    ptr_tree->root_ = delete_key_recursive(ptr_tree->root_, key);
}
```

There are many cases to consider.

```
node_t* delete_key_recursive(node_t* root, int key)

{

// code is here.

}
```

- If the tree is empty, then nothing to do. We cannot delete a node with a given key.

```
if(root == NULL)

{

        return NULL;

}
```

- If the key is less than the one at the root, call delete key on its left subtree

```
else if(key < root->key_)
  {
    root->left_ = delete_key_recursive(root->left_, key);
  }
```

- If the key is greater than the one at the root, call delete key on its right subtree

```
else if (key > root->key_)
{
    root->right_ = delete_key_recursive(root->right_, key);
}
```

- Now we have found the match for the key. We have consider 4 cases
    - Key at the leaf
    - Key at a node with no left subtree
    - Key at a node with no right subtree
    - Key at a node with both left and right subtree.

- Let us consider each case.
- **Key at the leaf.**

  This is the simplest case.

  ```
  if(root->left_ == NULL && root->right_ == NULL)
          {
                  free(root);
                  return NULL;
          }
  ```

  Remove the node and ground the pointer in its parent.
- **Key at a node with no left subtree**

  Attach its right subtree to its parent.

  ```
  else if(root->left_ == NULL)
          {
                  node_t* temp = root->right_;
                  free(root);
                  return temp;
          }
  ```
- **Key at a node with no right subtree**

  Attach its left subtree to its parent.

  ```
  else if(root->right_ == NULL)
          {
                  node_t* temp = root->left_;
                  free(root);
                  return temp;
          }
  ```

- **Key at a node with both left and right subtrees.**

  Find the inorder successor of what comes to the right.

```
node_t* inorder_successor(node_t* root)
{
    node_t* temp = root;
    while(temp->left_ != NULL)
        temp = temp->left_;
    return temp;
}
```

  Replace the key at the root with the successor.

  Modify the problem to remove this new key in the right subtree.

```
else
        {
            node_t* temp = inorder_successor(root->right_);
            root->key_ = temp->key_;
            root->right_ = delete_key_recursive(root->right_, temp->key_);
        }
```

  Check the code in the folder search_delete_key

How to store a tree and how to restore a tree?

If a list has to be stored, we could traverse and right the components to a file. We can restore the list by reading the components from the file and attaching them at the end of the list.

A list is a linear data structure. So we cannot do the way we have stated above.

A tree is non linear. A single traversal will not define the tree completely.

For example, if a tree of 3 nodes has an inorder traversal A B C, there are 5 possible trees which can produce the same inorder traversal.

An inorder and a postorder traversals of a tree uniquely defines a tree. Given the two traversals, we can reconstruct the tree.

Let us take an example to illustrate.



There are exactly 10 nodes:

A, B, C, D, E, F, G, H, I, J

The inorder traversal of the tree : D B H E A F C G J I

The post order traversal of the tree : D H E B F J I G C A

To reconstruct the tree, traverse the post order traversal from the right. The last node A has to be the root of the whole tree. Therefore, A is the root and A divides the inorder traversal into two parts. All nodes to the left of A are in left subtree of A and all the nodes to the right of A in the right subtree.

So, we know the root of the tree.

Next to the right of A in postorder traversal is C. C is to the right of A in inorder traversal. C is the root of the right subtree of A.

This way we may continue to build the whole tree.

The rest of it is left as an exercise for the readers.

Tree

A few points:

So far, we have discussed the concept of general trees, binary trees and binary search trees. We have discussed at length the interface of the binary search tree. We have also implemented a binary search tree using the node concept and dynamic memory. We shall discuss different ways of implementing the binary search tree in this unit and we will show one of these in detail.

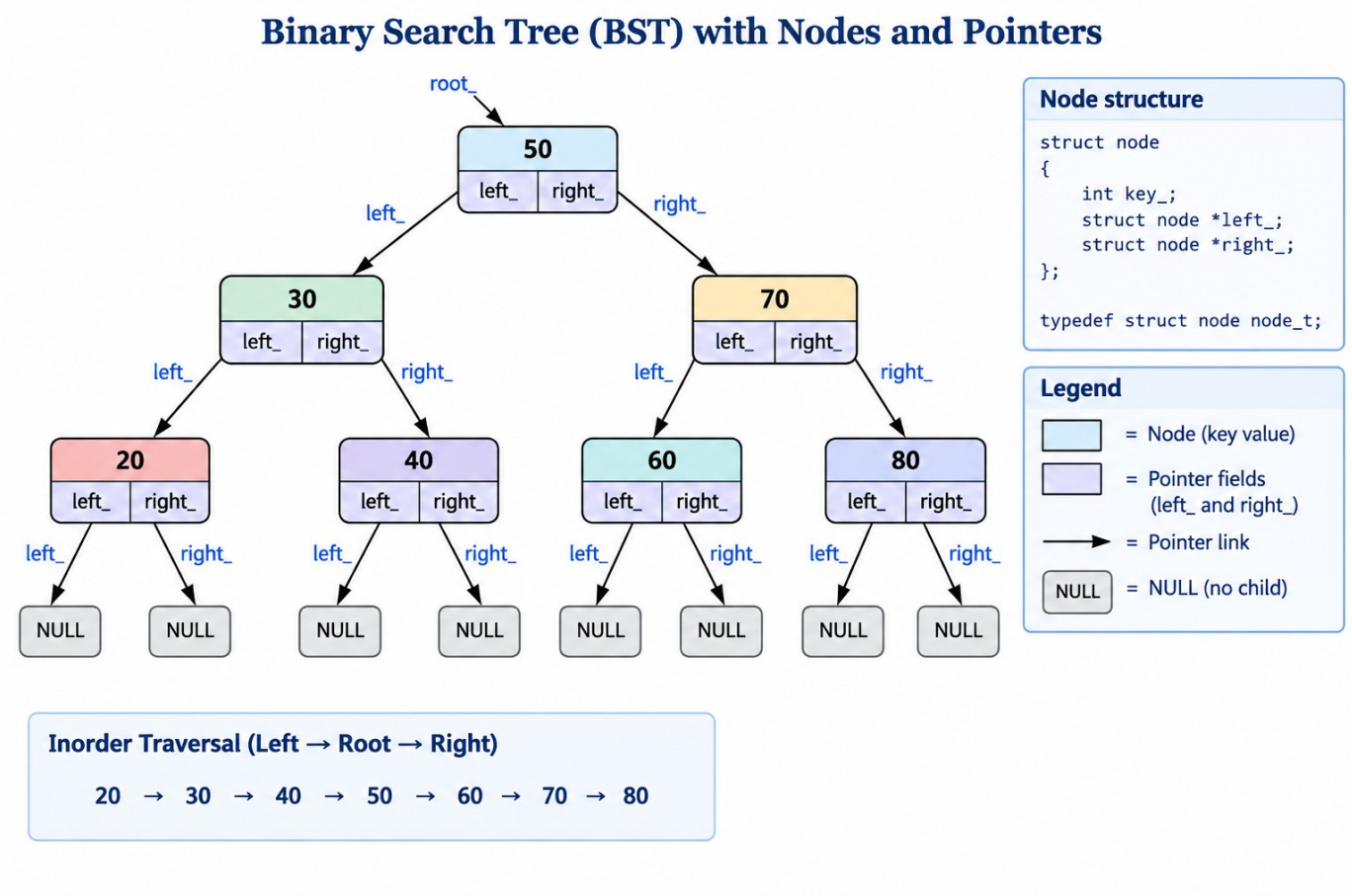
Implementation of binary search tree:

1. Node based implementation with dynamic memory.

The following structure was used in the discussion in the last lecture.

```
struct tree
{
    node_t *root_;
};
typedef struct tree tree_t;


struct node
{
    int key_;
    struct node *left_;
    struct node *right_;
};
typedef struct node node_t;
```



2. Array implementation of binary tree based on relating the parent and the two children.

   This implementation will only store the components. It does not store indices or pointers to the left subtree and the right subtree.

   We may or may not use the 0th location of the array. Depending on this, the mapping of the left subtree and the right subtree will vary.

   0 index based mapping :

   Parent : i , left child : 2i + 1; right child : 2i + 2

   1 index based mapping :

   Parent : i , left child : 2i ; right child : 2i + 1

   This arrangement is good only for complete binary trees where all nodes at higher levels are full and the lower level has all left nodes filled up.

   Otherwise, there will be lots of slots in the array will not be filled up.

Finding whether the slot is filled up would require a dummy value to be filled which will not be a valid value in the node.

This method is used while creating heap trees. We shall be discussing later in this course.

In this table, the key column represents the tree. This is 1 index based. The root has the index 1. The left child of 1 is 2 and the right child of 1 is 3. The left child of 2 is 4 and the right child of 2 is 5.

index	key
0	
1	50
2	30
3	70
4	20
5	40
6	60
7	80

3. Array implementation of binary tree with the node structure – each node has the element or the component, index of the left subtree and the index of the right subtree.

   root = 0

   The array content depends on the order in which the keys are inserted in this binary search tree. Assuming that the keys are given as 50, 30, 70, 20, 40, 60, 80, this will be the content of the array representing the tree.

   This implementation is discussed in detail in this chapter.

index	left	key	right
0	1	50	2
1	3	30	4
2	5	70	6
3	-1	20	-1
4	-1	40	-1
5	-1	60	-1
6	-1	80	-1

Whenever we get a new key, we must find a slot in the array to fill up. In our example, we have assumed that the next slot is available. But on deletion, slots may be available at different locations. To make this robust, we must implement a simple memory manager which keeps a list of free slots and gives a free slot when required and adds the slot to the list slot on deletion.

To make our example simple, we have not implemented the memory manager and also we have not implemented the deletion of a key from the tree.

Details of the implementation of a tree using an array of node structure:

Interface:

We are referring to the folder one. In this we refer to the header file tree.h to understand the structure of the tree.

The node structure is similar to the one implemented in the tree with dynamic allocation. The left and the right subtrees are indicated by indices and not pointers.

```
#define MAXSIZE 200
struct node
{
        int key_;
        int left_;
        int right_;
};
typedef struct node node_t;
```

```
struct tree
{
        int root_;
        int count_;
        node_t elem_[MAXSIZE];
};
typedef struct tree tree_t;
```

The tree has the root which shall be the index of the root element in the array. Initially, it will be -1 to indicate the tree is empty – not a valid index. Once an element is added, it becomes 0.

The count_ indicates the number of elements filled up. It is also used to find the next slot where the insertion can be done. In a robust implementation, the location should come from the free list.

The interface functions remain unchanged.

```
void init(tree_t *ptr_tree);

void deinit(tree_t *ptr_tree);

void disp(tree_t *ptr_tree);

void insert(tree_t *ptr_tree, int key);
```

Implementation:

This code is from the folder one and the file tree.c.

Initialization and Deinitialization are similar. The tree is made empty and the number of elements is made 0.

```
void init(tree_t *ptr_tree)
{
        ptr_tree->root_ = -1;
        ptr_tree->count_ = 0;
}
```

```
void deinit(tree_t *ptr_tree)
{
	ptr_tree->root_ = -1;
	ptr_tree->count_ = 0;
}
```

Display:

This is an inorder traversal of the tree.

The disp function calls the disp_recursive function with the elem array and the root.

Observe unlike the normal way of handling an array, the size of the array is not passed as argument.

You may want to reason think of the reason for this!

The recursive function checks if the tree is empty. If not, calls the same function recursively on its left subtree, displays the key, then calls the function recursively on its right subtree.

```
static void disp_recursive(node_t elem[], int temp)
{
	if(temp != -1)
	{
		disp_recursive(elem, elem[temp].left_);
		printf("%d ", elem[temp].key_);
		disp_recursive(elem, elem[temp].right_);
	}
}
```

```
void disp(tree_t *ptr_tree)
{
        printf("Tree : ");
        disp_recursive(ptr_tree->elem_, ptr_tree->root_);
        printf("\n");
}
```

Insert:

The insert function checks whether the array is full. If we have a memory manager, then the list of free slots should indicate whether the tree is full.

The count will indicate the slot where the key can be inserted. Then the count is incremented to indicate the number of elements after insertion. The left and the right of the new node are always grounded.

Then, insert calls the recursive routine to insert this new node into the tree. This function passes the array of nodes, the root and the index where the new node has been inserted.

```
void insert(tree_t *ptr_tree, int key)
{
    if(ptr_tree->count_ == MAXSIZE)
    {
            printf("tree full\n");
            exit(1);
    }
    int temp = ptr_tree->count_++;
    ptr_tree->elem_[temp].key_ = key;
    ptr_tree->elem_[temp].left_ = ptr_tree->elem_[temp].right_ = -1;

    ptr_tree->root_ = insert_recursive(ptr_tree->elem_, ptr_tree->root_, temp);
}
```

If the tree is empty, the index of the new node is the root of the tree.

Otherwise, based on the key, insert in the left subtree or the right subtree.

```
static int insert_recursive(node_t elem[], int root, int temp)
{
	if(root == -1)
	{
		root = temp;
	}
	else if(elem[root].key_ > elem[temp].key_)
	{
		elem[root].left_ = insert_recursive(elem, elem[root].left_, temp);
	}
	else
	{
		elem[root].right_ = insert_recursive(elem, elem[root].right_, temp);
	}
	return root;
}
```

This function is very similar to the function we wrote for tree with dynamic allocation.

Let us examine a few more interface functions for this tree implementation.

These are found in folder **two**.

These functions are in tree.h.

```
// 1. find # of nodes

int number_of_nodes(tree_t *ptr_tree);

// 2. find # of leaves

int number_of_leaves(tree_t *ptr_tree);
```

The corresponding function implementation are in tree.c.

To find the number of nodes, this function number_of_nodes calls the corresponding recursive function with the elem array and the root.

```
int number_of_nodes(tree_t *ptr_tree)
{
	return number_of_nodes_recursive(ptr_tree->elem_, ptr_tree->root_);
}
```

This recursive function returns 0 if the tree is empty.
Otherwise, this function counts the root as 1 and add to it the number of nodes in the left subtree and the number of nodes in the right subtree.

```
int number_of_nodes_recursive(node_t elem[], int temp)
{
	if(temp == -1)
	{
		return 0;
	}
	else
	{
		return 1 + number_of_nodes_recursive(elem, elem[temp].left_)
			+ number_of_nodes_recursive(elem, elem[temp].right_);
	}
}
```

To find the number of leaves, this function calls the corresponding recursive function with the array elem and the root.

```
int number_of_leaves(tree_t *ptr_tree)
{
	return number_of_leaves_recursive(ptr_tree->elem_, ptr_tree->root_);
}
```

This recursive function returns 0 if the tree is empty, returns 1 if the root is a leaf else
Returns the sum of the number of leaves in the left subtree and the number of leaves in the right subtree.

```
int number_of_leaves_recursive(node_t elem[], int temp)
{
	if(temp == -1)
	{
		return 0;
	}
	else if(elem[temp].left_ == -1 && elem[temp].right_ == -1)
	{
		return 1;
	}
	else
	{
		return number_of_leaves_recursive(elem, elem[temp].left_)
			+ number_of_leaves_recursive(elem, elem[temp].right_);
	}
}
```

Let us examine two more functions – copy and compare trees.
These are in the folder **three**. The files are tree.h and tree.c.

The interface for copy and compare have not changed from the previous lectures.

```
// 3. copy tree
void copy(tree_t* ptr_src, tree_t* ptr_dst);
// 4. compare tree
int are_same(tree_t* tree1, tree_t* tree2);
```

Compare two trees:

This function are_same in turn invokes the corresponding recursive function. This function passes the array elem and root of both the trees.

```
int are_same(tree_t* ptr_tree1, tree_t* ptr_tree2)
{
	return are_same_recursive(ptr_tree1->elem_, ptr_tree1->root_,
		ptr_tree2->elem_, ptr_tree2->root_);
}
```

This recursive function compares the roots. If both the trees are empty, then the trees are same.
If one of the trees is empty, then the trees are different. Otherwise compare the keys and compare the left subtrees and right subtrees recursively.

```
int are_same_recursive(node_t elem1[], int root1, node_t elem2[], int root2)
{
	if(root1 == -1 && root2 == -1)
		return 1;
	else if(root1 == -1 || root2 == -1)
		return 0;
	else
		return elem1[root1].key_ == elem2[root2].key_ &&
			are_same_recursive(elem1, elem1[root1].left_,
				elem2, elem2[root2].left_)  &&
			are_same_recursive(elem1, elem1[root1].right_,
				elem2, elem2[root2].right_);
}
```

Copy the tree:

This function is made into iterative one. As we are dealing with arrays, we do not have always use recursion. It might be easier to right a non-recursive function.
This function copies the root, the count and the array elements.

We could debate whether we could stop when I reaches the count or we should copy till MAXSIZE!

```
void copy(tree_t* ptr_src, tree_t* ptr_dst)
{
	ptr_dst->root_ = ptr_src->root_;
	ptr_dst->count_ = ptr_src->count_;
	for(int i = 0; i < MAXSIZE; ++i)
	{
		ptr_dst->elem_[i] = ptr_src->elem_[i];
	}
}
```

Can you make this recursive similar to insert or compare functions?
This is left to you as an exercise.

Tree Traversals:

We normally traverse a tree inorder, preorder and postorder.

These functions listed here are very similar to the ones we wrote for tree with dynamic allocation.

```
static void disp_inorder_recursive(node_t elem[], int temp)
{
	if(temp != -1)
	{
		disp_inorder_recursive(elem, elem[temp].left_);
		printf("%d ", elem[temp].key_);
		disp_inorder_recursive(elem, elem[temp].right_);
	}
}
static void disp_preorder_recursive(node_t elem[], int temp)
{
	if(temp != -1)
	{
		printf("%d ", elem[temp].key_);
		disp_preorder_recursive(elem, elem[temp].left_);
		disp_preorder_recursive(elem, elem[temp].right_);
	}
}

static void disp_postorder_recursive(node_t elem[], int temp)
{
	if(temp != -1)
	{
		disp_postorder_recursive(elem, elem[temp].left_);
		disp_postorder_recursive(elem, elem[temp].right_);
		printf("%d ", elem[temp].key_);
	}
}
```

```
void disp_inorder(tree_t *ptr_tree)
{
	printf("Tree inorder: ");
	disp_inorder_recursive(ptr_tree->elem_, ptr_tree->root_);
	printf("\n");
}

void disp_preorder(tree_t *ptr_tree)
{
	printf("Tree preorder : ");
	disp_preorder_recursive(ptr_tree->elem_, ptr_tree->root_);
	printf("\n");
}

void disp_postorder(tree_t *ptr_tree)
{
	printf("Tree postorder: ");
	disp_postorder_recursive(ptr_tree->elem_, ptr_tree->root_);
	printf("\n");
}
```

With this, we come to the end of discussion of tree implemented using an array of nodes.

Expression Tree

A few points:

In this chapter, we will examine how a postfix expression can be converted to an expression tree where the leaves are operands and internal nodes are operators. We will also examine how we can find the value of such an expression.

Algorithm:
Input : an expression in postfix. For simplicity, the operands are assumed to be single digits.
Process : construct an expression tree
Output: find the value of the expression

Phase 1:
Construct the expression tree:

Data structure used: stack of tree nodes

Scan the input left to right
If operand, create a tree node and push it to a stack of tree nodes.
If operator, pop two tree nodes from the stack, create a new tree node with the operator as the key.
Attach the popped nodes as subtrees of the new node.
Push the new node into the stack.

Phase 2 :
Evaluate the expression tree:
Stack should have one and only one tree node.
On this tree, do the evaluation.
If the node is a leaf, return the operand.
Else
    Evaluate the left subtree
    Evaluate the right subtree
    Apply the operator of this node on these two results

Details of the interface and the implementation:

Stack:

We are using an array implementation of the stack. Each element of the stack is a node which has two self referential members left_ and right_ standing for left and right subtrees.
It has a key field of char type.

This design has limitation that the operand can be one character. To avoid this, you may want to design a union of operand(int) and operator(char). You may not require a discriminator as the operands are in the leaves alone.

Stack structure:

```
#define MAXSIZE 100
struct node
{
        char ch_;
        struct node *left_;
        struct node *right_;
};
typedef struct node node_t;

struct stack
{
        node_t* key_[MAXSIZE];
        int top_;
};
typedef struct stack stack_t;
```

stack interface:
We do observe a few changes compared to a normal stack.

Observe that pop returns the key of the stack which is a pointer to a node.

The function push_operand is similar to the push we have had earlier.

The function push_operator requires that two operands be popped, a new node created and these Two popped nodes be added as subtrees. So, we have made a separate function for pushing operators.

```
void init_stack(stack_t *ptr_stack);
void deinit_stack();
void push_operand(stack_t *ptr_stack, char ch);
void push_operator(stack_t *ptr_stack, char ch);
node_t* pop(stack_t *ptr_stack);
int is_empty(stack_t *ptr_stack);
int is_full(stack_t *ptr_stack);
```

The eval function pops the one and only node from the stack.
Evaluates the left subtree recursively and the right subtree recursively and then combine the results based on the operator at the root.

```
int eval(stack_t *ptr_stack);
```

stack implementation:
create_node:
This helper function creates a node. If there is no memory, this function gives a message and exits the program.

```
static node_t* create_node(char ch)
{
	node_t *temp = (node_t*)malloc(sizeof(node_t));
	if(temp == NULL)
	{
		printf("memory full\n"); exit(1);
	}
	temp->ch_ = ch;
	temp->left_ = temp->right_ = NULL;
	return temp;
}
```

push_operand:
This function makes a node and pushes the node on to the stack.
The operand is stored as a single character.

```
void push_operand(stack_t *ptr_stack, char ch)
{
	if(! is_full(ptr_stack))
	{
		node_t* temp = create_node(ch);
		ptr_stack->key_[++ptr_stack->top_] = temp;
	}
	else
	{
		printf("stack full; cannot push\n");
	}
}
```

push_operator:

In this function, a new node is created with the operator as the key.
We pop two nodes from the stack and make them right and left subtrees of the new node created.
We then push the new node into the stack.

```
void push_operator(stack_t *ptr_stack, char ch)
{
	if(! is_full(ptr_stack))
	{
		node_t* temp = create_node(ch);
		temp->right_ = pop(ptr_stack);
		temp->left_ = pop(ptr_stack);
		ptr_stack->key_[++ptr_stack->top_] = temp;
	}
}
```

pop:
This function gives a message and returns NULL character if the stack is empty.
Otherwise returns the top element from the stack.
Observe this function returns a pointer to a node.

```
node_t* pop(stack_t *ptr_stack)
{
	if(! is_empty(ptr_stack))
	{
		return ptr_stack->key_[ptr_stack->top_--];
	}
	else
	{
		printf("stack empty; cannot pop\n");
		return '\0';
	}
}
```

Evaluation:

The eval function calls eval_recursive with the root of the tree – which is obtained by popping from the stack.
If the node is a leaf corresponding to the operand, the numeric value of the digit is returned.
This function in turn calls the evaluation of left and right subtrees recursively.
Once the results are obtained, based on the operator, the corresponding operation is performed.

```
int eval_recursive(node_t* root)
{

   int left, right;

   if (root->left_ == NULL && root->right_ == NULL)
      return root->ch_ - '0';

   left = eval_recursive(root->left_);
   right = eval_recursive(root->right_);

   switch (root->ch_)
   {
      case '+': return left + right;
      case '-': return left - right;
      case '*': return left * right;
      case '/': return left / right;
   }

   return 0;
}

int eval(stack_t *ptr_stack)
{
         return eval_recursive(pop(ptr_stack));
}
```

With this, the expression tree and its evaluation are complete.

Tree

A few points:

So far, we have discussed general tree, binary tree and binary search tree. We have also discussed at length two different implementations of the tree – node and dynamic memory, node and array based.

In this discussion, we will focus on two aspects.

a) Make better use of pointers in node and dynamic memory-based tree structure.
b) Evaluate an expression by creating an expression tree based on an expression in postfix notation.

Threaded binary tree:

In a tree of n nodes, there are only n-1 edges or branches. In a node based implementation, each node has 2 pointers. We require only n-1 pointers to point to nodes – these are called structural pointers. So, as many as n + 1 pointers will be grounded.

Based on a particular selected order of traversal (say inorder), can we, instead of grounding the left pointer, make it point to the predecessor? Similarly, instead of grounding the right pointer, can we make it point to the successor? Such a pointer is called a thresad.

How do we make out whether the pointer is a structural pointer or a thread?

To facilitate this, we may require two bit fields.

Check the structure of the node in the header file : tree.h in the folder threaded.

Lthread is 1 if left is thread and a structural pointer. Similarly, rthread is 1 if right is a thread and not a structural pointer. Otherwise, they will be 0.

Interface:

```
struct node
{
        int key_;
        struct node *left_;
        struct node *right_;
        int lthread_;
        int rthread_;
};
typedef struct node node_t;
```

```
struct tree
{
        node_t *root_;
};
typedef struct tree tree_t;
```

The interface of the tree has not changed.

```
void init(tree_t *ptr_tree);

void deinit(tree_t *ptr_tree);

void disp(tree_t *ptr_tree);

void insert(tree_t *ptr_tree, int key);
```

Implementation:

Initialize :

The root is grounded.

```
void init(tree_t *ptr_tree)
{
        ptr_tree->root_ = NULL;
}
```

Insert:

There is a helper function which creates a node.

Note that a new node is created. If the allocation fails, this function gives an error message and exits the program.
To the new node, key is copied.
Always the node is added as a leaf. The pointers are grounded.
We assume that the pointers are threads. We could also think otherwise and suitably take care in the insert routine.

```
static node_t* create_node(int key)
{
        node_t *temp = (node_t*)malloc(sizeof(node_t));
        if(temp == NULL)
        {
                printf("no more memory\n"); exit(1);
        }
        temp->key_ = key;
        temp->left_ = temp->right_ = NULL;
        temp->lthread_ = temp->rthread_ = 1;
        return temp;
}
```

This insert function calls the function insert_key to create a node and insert the node and adjust the structural and thread pointers.

```
void insert(tree_t *ptr_tree, int key)
{
        ptr_tree->root_ = insert_key(ptr_tree->root_, key);
}
```

This is quite a long function.
We will discuss the different parts at length.

```
static node_t* insert_key(node_t* root, int key)
{
        node_t *temp = create_node(key);
        node_t *parent = NULL;
        node_t *curr = root;
         while (curr != NULL)
         {
               parent = curr;
               if (key < curr->key_)
               {
                  if (curr->lthread_ == 0)
                              curr = curr->left_;
                  else
                              break;
               }
              else
              {
                   if (curr->rthread_ == 0)
                              curr = curr->right_;
                   else
                              break;
```

```
            }
        }
    if (parent == NULL)
    {
       return temp;
    }

    /* Insert as left child */
    if (key < parent->key_)
    {
       temp->left_ = parent->left_;
       temp->right_ = parent;

       parent->left_ = temp;
       parent->lthread_ = 0;
    }

    /* Insert as right child */
    else
    {
       temp->right_ = parent->right_;
       temp->left_ = parent;

       parent->right_ = temp;
       parent->rthread_ = 0;
    }

    return root;
}
```

- Create a node and initialize local variables
  Observe that the node does not have a parent pointer. It is a local variable.
  ```
  node_t *temp = create_node(key);
  node_t *parent = NULL;
  node_t *curr = root;
  ```

- Take care of the base case when the first node is added and the tree is empty.
  ```
  if (parent == NULL)
  {
     return temp;
  }
  ```

- This is the loop to find the position to insert in the binary search tree. When the subtree is not empty, move along the tree. Remember that we always attach a new node as a leaf. Update the

parent before changing current. Based on the key comparison, decide to move left or right. If we have reached a thread, it means that we have reached a leaf.
Removing the break in the code is left as an exercise to you.

```
while (curr != NULL)
{
      parent = curr;
      if (key < curr->key_)
      {
         if (curr->lthread_ == 0)
               curr = curr->left_;
         else
                break;
      }
      else
       {
               if (curr->rthread_ == 0)
                        curr = curr->right_;
               else
                        break;
       }
  }
```

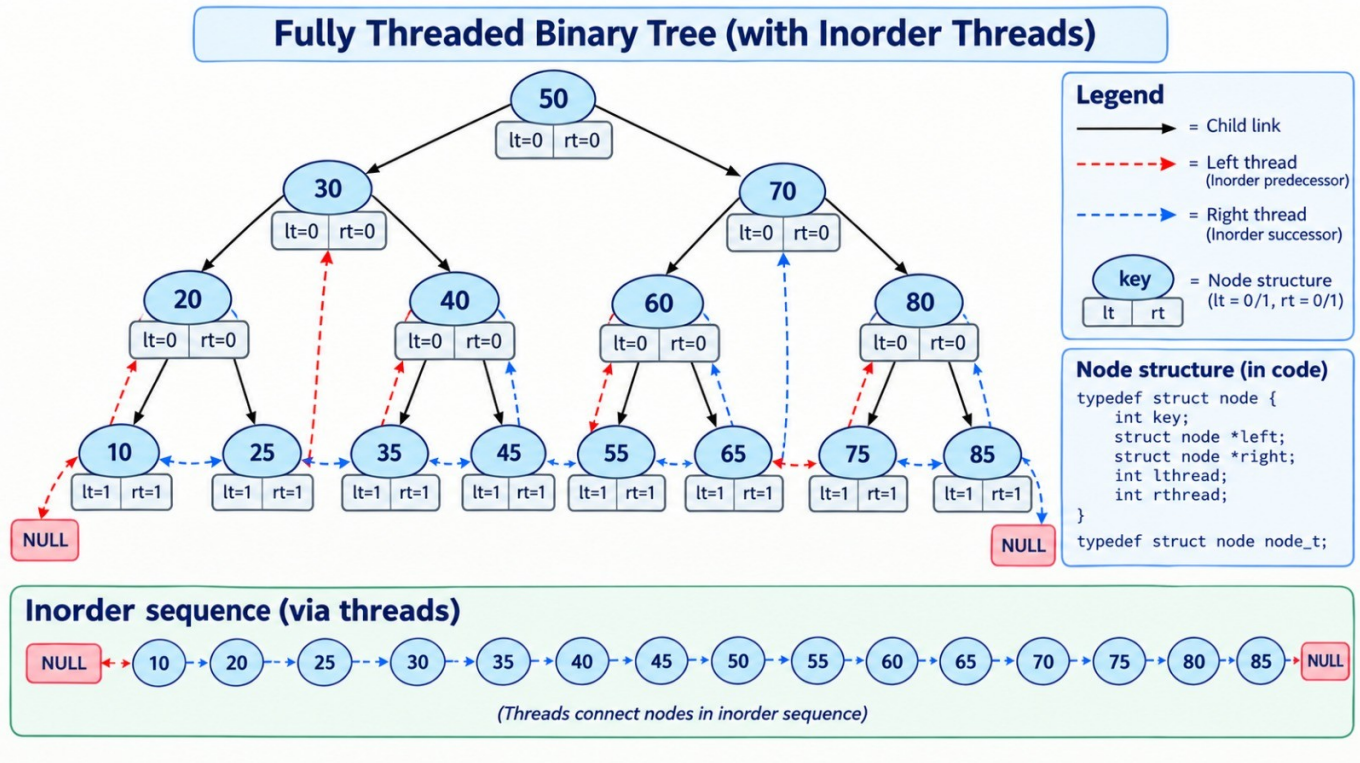
- Once the loop is exited, we have found a position to insert. We should find whether the insertion is happening in the left subtree or the right subtree with respect to the parent.
  To decide that, key is compared with the key of the parent.

```
  if (key < parent->key_)
  {
     temp->left_ = parent->left_; // why temp->left_ not NULL?
     temp->right_ = parent; // parent is the successor; right_ is a thread

     parent->left_ = temp; // attach to the left subtree; so this is a structural pointer
     parent->lthread_ = 0; // and hence not a thread
  }
  else
  {
     temp->right_ = parent->right_; // why temp->right_ not NULL?

     temp->left_ = parent; // parent is the successor; left_ is a thread


     parent->right_ = temp; // attach to the right subtree; so this is a structural pointer
     parent->rthread_ = 0; // and hence not a thread
  }
  return root;
```



The left of the leftmost node and the right of the rightmost node are threads and may point to the root node or may be grounded.

Display the tree in inorder:

This function disp calls the inorder function. Note that the inorder function is not recursive.

```
void disp(tree_t *ptr_tree)
{
        printf("Tree : ");
        inorder(ptr_tree->root_);
        printf("\n");
}
```

There is a helper function to find the leftmost node in the tree.
If the tree is empty, there is no leftmost node.
Otherwise keep traversing left. Remember no pointers are grounded. So, we will come to know that the left subtree is empty if the left is a thread.

```
static node_t *leftmost(node_t *temp)
{
   if (temp == NULL)
      return NULL;

   while (temp->lthread_ == 0)
      temp = temp->left_;

   return temp;
}
```

This function inorder finds the leftmost node without a left subtree.
It will display the key and then follow the right if is is a thread.
Otherwise it is a structural pointer. Take one step right and then again find the leftmost node.

```
static void inorder(node_t *root)
{
   node_t *temp = leftmost(root);

   while (temp != NULL)
   {
      printf("%d ", temp->key_);

      if (temp->rthread_ == 1)
         temp = temp->right_;
      else
         temp = leftmost(temp->right_);
   }
}
```

This completes the discussion of threaded binary tree.

Heap

A few points:

There are many words in Computer Science which have different meanings depending on the context. Already we have observed that thread in Operating Systems stands for a unit of execution whereas threaded tree makes the NULL structural pointers point to its successor or predecessor making iterative functions possible for a particular order of traversal.

Similarly, the word heap has 2 meanings. Heap in memory management refers to a part of the memory supporting allocation and deallocation of memory dynamically.

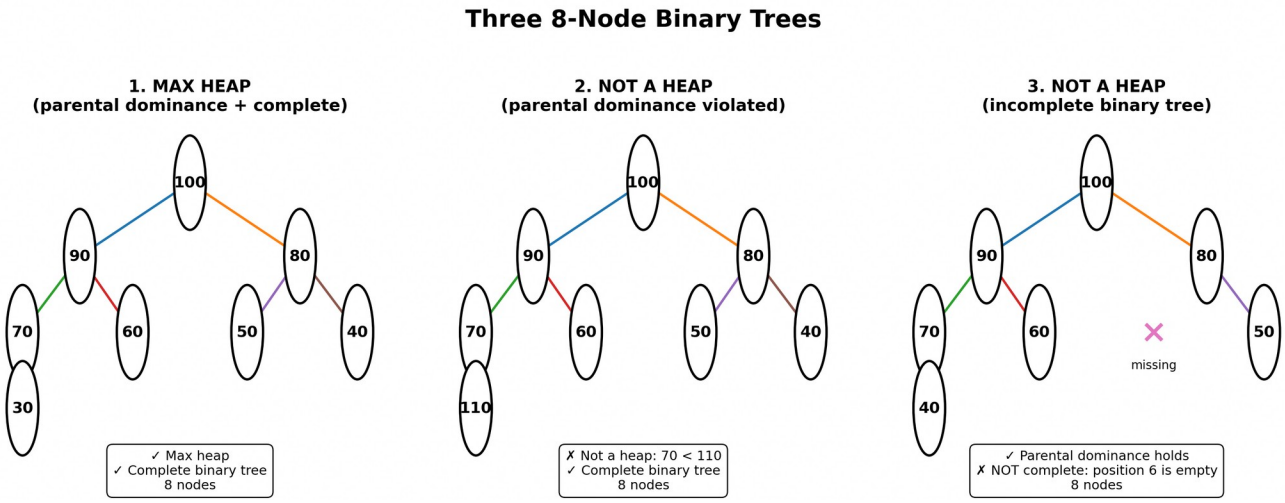
In data structure, a heap refers to a special type of binary tree. It is not a search tree.
There are two kinds of heaps – called maxheap and minheap.

In our discussion, we will discuss the maxheap. The interface and the implementation of minheap will be similar – we have to change the comparison operator from “greater than” to “less than”.

A heap(maxheap) is a tree with two special properties.

1. Complete binary tree:
   The nodes are filled up at each level.
   At the bottom most level, leftmost nodes are filled up.
2. Parental dominance:
   The key at the root is greater than the keys of its subtrees recursively.

Three diagrams in the next page illustrate the properties of the heap tree.
The first one is a heap. The root has the highest value and keys at the roots of the subtree are less than that of the root. There is no ordering of keys at the left and the right subtrees. This must be true recursively. Evidently, leaves are always heaps. Also, observe that the first 3 levels are totally filled up. In the last level, the leftmost nodes are filled up.



The second example is not a heap as the child of node with the key 70 has the key 110. It does not satisfy parental dominance. So, this is not a heap.

In the third example, the second level(counting from 0) is not completely filled. The left subtree of node with key 80 is missing. So this is not a heap. This tree has 7 nodes.

Implementation of a heap tree:

We always implement a heap tree using an array. As the tree is a complete tree, it provides an easy of mapping to an array and also provides simple ways by which we can make out whether the tree is full or empty, insert, traverse and so on.

We have two ways of mapping a heap to an array.

1. We may start with the root being mapped to 0th element of the array.

   In such a case, we use the following relationships to relate the root, the left subtree and the right subtree.

   Leftchild(i) = 2i + 1

   Rightchild(i) = 2i + 2

   Parent(i) = floor((i-1)/2)

2. We may start with the root being mapped to 1st element of the array.

   In such a case, we use the following relationships to relate the root, the left subtree and the right subtree.

   Leftchild(i) = 2i

   Rightchild(i) = 2i + 1

   Parent(i) = floor(i/2)

In our discussion, we will use the second implantation.

Advantages of heap implementation:

- No pointers
- No extra storage
- Easy way to determine empty or full
- Simple way to traverse either as an array or as a tree
- Does not waste memory as the tree is a complete binary tree

Uses of Heap :

1. Heapsort : There is way of sorting using the concept of heap. This is theoretically one of the most efficient algorithm. It is as efficient as any other algorithm for sorting based on comparison. But, this algorithm is not cache friendly. So, no library of any programming language implements this algorithm.
2. Finding maximum (or minimum) repeatedly:

   In many applications, we may have insert and delete based on the order of the keys. The classical example is priority queue. We enqueue a task and then dequeue a task based on priority. In such cases, both these operations are equally efficient and every one of them requires less number of operations compared to the number of elements in the tree. In fact, the number of operations is proportional to the logarithm to base 2 of the number of elements in the tree.

Example 1 :

Build a heap:

Input : array having n elements

Output : array representing a maxheap of n elements

Structure of heap:

This heap has an array of keys and has the number of elements in use.

```
struct heap
{
    int key_[MAXSIZE];
    int n_;
};
typedef struct heap  heap_t;
```

Interface of heap :

```
void init(heap_t *ptr_heap);

void deinit(heap_t *ptr_heap);

void disp(heap_t *ptr_heap);


void build_heap(heap_t *ptr_heap, int a[], int n);
```

Implementation:

- **Initialize and deinitialize:**

  There is no dynamic allocation. The number of elements in the tree is made 0

  ```
  ptr_heap->n_ = 0;
  ```

- **Display**

  This function displays the element of the tree as an array varying index from 1 to n.

Build a heap:

This takes an array as argument and copies the array to the heap data structure and converts it to a heap.

There are 2 parts in the algorithms.

The first part copies the given array into the array in the heap structure.

The second part tries to convert every subtree of this array representation to a heap.

All leaves are heaps. So, the conversions start from the middle of the array skipping all leaves and moves towards the root.

The function heapify converts the tree rooted at position i.

We will discuss heapify in the next section.

```
void build_heap(heap_t *ptr_heap, int a[], int n)
{
    ptr_heap->n_ = n;
    for(int i = 1; i <= n; ++i)
    {
            ptr_heap->key_[i] = a[i];
    }
    for(int i = n / 2; i >= 1; --i)
    {
            heapify(ptr_heap->key_, ptr_heap->n_, i);
            disp(ptr_heap);
    }
}
```

Heapify:

Given that the left subtrees and the right subtrees of a node are already heap, we want to position the root element properly in the subtrees so that the tree rooted at the given root becomes a heap.

The position of the root is i.

Its left child is at position 2i. This is stored in the variable j.

The value at the root i is copied so that the ith location becomes a hold – can be modified.

We enter a loop when j is still a valid index(j <= n) and the root(ith) element is not greater than the keys at the roots of its subtrees(not a heap yet).

If j+1 key exists and is greater than the jth key, we move to the right by incrementing j.

If jth key is less than the value of the root, out search is over.

Otherwise, move the jth subtree by shifting the hold and changing i and j.

When we exit the loop, we copy the val to the ith location.

```
static void heapify(int a[], int n, int pos)
{
    int i = pos;
    int j = 2 * i;
    int val = a[i];
    int is_heap = 0;
    while(j <= n && ! is_heap)
    {
        if(j + 1 <= n && a[j+1] > a[j])
            ++j;
        if(a[j] > val)
        {
            a[i] = a[j];
            i = j;
            j = 2 * i;
        }
        else
        {
            is_heap = 1;
        }

    }
    a[i] = val;
}
```

Example 2:

Build a menu driven program to support insertion and deletion:

In this example, a few functions are added.

Interface:

```
void insert(heap_t *ptr_heap, int key);
int delete_max(heap_t *ptr_heap);
int peek_max(heap_t *ptr_heap);
```

insert a key:

In insert, we always add the element at the end of the array and increase the heap size by 1. Then we propagate the element up towards the root to make the tree a heap.

```
void insert(heap_t *ptr_heap, int key)
{
    // add at the end
    // heapify upwards
    int pos = ++ptr_heap->n_;
    if(pos > MAXSIZE)
    {
            printf("heap full\n"); exit(1);
    }
    ptr_heap->key_[pos] = key;
    while(pos > 1 && ptr_heap->key_[pos] > ptr_heap->key_[pos/2])
    {
            swap(&ptr_heap->key_[pos], &ptr_heap->key_[pos/2]);
            pos = pos / 2;
    }
}
```

The key is added at the end. The number of elements in the tree is incremented.

If the key added is greater than that of its parent, we interchange the two and repeat the operation until the above condition becomes false or we reach the root.

In heapify routine, we propagated a hole. In this we have used the concept of swapping instead.

Delete key:

In delete, we always delete the max element which is at the root. We overwrite this root element with the last element of the array, decrease the tree size by 1 and then heapfiy about the root.

```
int delete_max(heap_t *ptr_heap)
{
    if(ptr_heap->n_ == 0)
    {
            printf("heap empty\n"); exit(1);
    }
    int res = ptr_heap->key_[1];
    ptr_heap->key_[1] = ptr_heap->key_[ptr_heap->n_];

    heapify(ptr_heap->key_, --ptr_heap->n_, 1);
    return res;
}
```

Peek max :

This will return the value at the root – the key at index 1.

```
int peek_max(heap_t *ptr_heap)
{
    return ptr_heap->key_[1];
}
```

Observe that in delete, we call heapify, which discards one half the tree each time by doubling the value of j.

Observe that in insert, we keep moving towards the parent and do not move towards the sibling by halving the index each time.

So, in both the cases, the number of operations we do is proportional to the log to base 2 of number of elements in the tree .

Example 3:

Priority queue using heap:

Structure of priority queue:

```
struct node
{
    int key_;
    struct node* next_;
};
typedef struct node node_t;

struct queue
{
    heap_t q_;
};
typedef struct queue queue_t;
```

The priority queue has composition of heap in it.

Interface of priority queue:
All these functions will in turn delegate the functionality to those of the heap.

```
void init_queue(queue_t *ptr_queue);
void deinit_queue(queue_t *ptr_queue);
void enqueue(queue_t *ptr_queue, int key);
int dequeue(queue_t *ptr_queue);
int is_empty(queue_t *ptr_queue);
int is_full(queue_t *ptr_queue);
```

Implementation of priority queue:

```
void init_queue(queue_t *ptr_queue)
{
    init(&ptr_queue->q_);
}

void deinit_queue(queue_t *ptr_queue)
{
    deinit(&ptr_queue->q_);
}



void enqueue(queue_t *ptr_queue, int key)
{
    insert(&ptr_queue->q_, key);

}
int dequeue(queue_t *ptr_queue)
{
    return delete_max(&ptr_queue->q_);
}
```

Here, we end the discussion of heap.

Please note that the sorting based on heap – heapsort is never used in any library.

Also, whenever insertion and deletion are repeatedly executed and are based on a key, heap the most preferred data structure.

AVL Trees

A few points:

A binary search tree allows us to discard one of the subtrees while searching. Based on the key comparison at the root, we could discard the left subtree or the right subtree. Would that mean that the number of comparisons in the worst case will be log(n, 2) where n is the number of nodes in the tree?

The answer is NO. It is possible that the tree could become skewed. The tree could degenerate into a list. In such a case, the number of comparisons while searching will be proportional to the number of elements and will not become logarithmic.

How do we make the tree have the same depth – at most with a difference of 1 – no matter in which direction we move from the root?

This leads to a concept called Height Balanced Binary Search Trees. AVL tree is one of them.

AVL trees:

This tree was invented by two Russian Mathematicians : Adelson-Velsky and Landis.

An **AVL tree** is a **self-balancing Binary Search Tree (BST)**.

AVL stands for **Adelson-Velsky and Landis**, the two researchers who introduced the tree.

The key property is:

For every node, the heights of its left and right subtrees differ by at most 1.

Balance Factor

For every node: h: stands for the height

$$\text{Balance Factor} = h(\text{left subtree}) - h(\text{right subtree})$$

For an AVL tree, the balance factor must be:

$$-1,\ 0,\ +1$$

If the balance factor becomes:

+2 → left-heavy → imbalance

-2 → right-heavy → imbalance

we need a **rotation**.

Example

```
    30
   / \
  20  40
```

If the heights of the left and right subtrees are both 1:

$$BF(30) = 1 - 1 = 0$$

So the node is balanced.

If the balance factor becomes 2 or -2, then we rotate the tree such that the balance factor is 1 or 0 or -1.

For example :

```
    30
   /
  20
 /
10
```

At node 30:

$BF(30) = 2$

After rotation:

```
    20
   / \
  10  30
```

Now:

BF(20) = 0

and the tree is balanced.

Four cases of rotation:

There are four possible imbalance configurations:

Case	Configuration	Rotation
LL	Left-Left	Right rotation
RR	Right-Right	Left rotation
LR	Left-Right	Left rotation + Right rotation
RL	Right-Left	Right rotation + Left rotation

A very useful memory rule is:

Same direction → single rotation
Opposite directions → double rotation

---

1. **LL Rotation**

Example

Insert:

30, 20, 10

The resulting BST is:

```
      30
     /
    20
   /
```

10

The path from the unbalanced node is:

30 → 20 → 10

Both moves are **left**.

Therefore, this is an **LL case**.

Balance factor

At node 30:

Left height = 2

Right height = 0

Therefore:

$$BF(30) = 2$$

The tree is left-heavy.

Solution: Right Rotation

Rotate right around 30:

```
      30
     /
    20
   /
  10
```

```
    ↓
```

```
    20
   / \
  10  30
```

The tree is now balanced.

LL rule

LL → Right Rotation

---

2. **RR Rotation**

Now consider inserting:

10, 20, 30

The BST becomes:

```
  10
   \
    20
     \
      30
```

The path is:

10 → 20 → 30

Both moves are **right**.

Therefore, this is an **RR case**.

At node 10:

$$BF(10) = 0 - 2 = -2$$ The node is right-heavy.

Solution: Left Rotation

Rotate left around 10:

```
  10
   \
    20
     \
      30
       ↓
     20
    / \
  10   30
```

RR rule

RR → Left Rotation

---

3. **LR Rotation**

This is slightly more interesting.

Insert:

30, 10, 20

The BST becomes:

```
    30
   /
  10
   \
    20
```

The path is:

30 → 10 → 20

The first move is **left**, but the second is **right**.

Therefore:

Left → Right

This is an **LR case**.

Step 1: Left rotation

First rotate the subtree rooted at 10 to the left.

```
    30
   /
  10
   \
```

```
      20
```

becomes:

```
      30
     /
    20
   /
  10
```

Now the tree has become an LL configuration.

Step 2: Right rotation

Rotate right around 30:

```
      30
     /
    20
   /
  10

    ↓

     20
    / \
  10   30
```

The tree is balanced.

LR rule

LR → Left Rotation

followed by

Right Rotation

---

4. **RL Rotation**

Now consider:

10, 30, 20

The BST becomes:

```
  10
   \
    30
   /
  20
```

The path is:

10 → 30 → 20

The first move is **right**, followed by **left**.

Therefore, this is an **RL case**.

Step 1: Right rotation

Rotate right around 30:

```
  10
   \
    30
   /
  20
```

becomes:

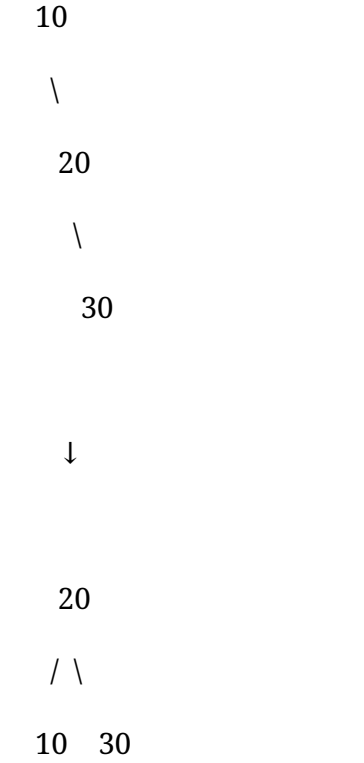
```
  10
   \
    20
      \
```

30

Now we have an RR configuration.

Step 2: Left rotation

Rotate left around 10:



The tree is balanced.

RL rule

RL → Right Rotation

followed by

Left Rotation

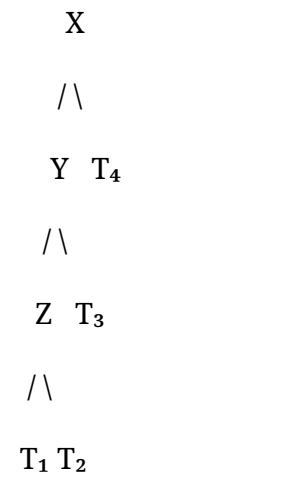
A few points:

- Balancing does not alter the property of the tree. The tree remains to be a BST.
- Rotations changes the shape; but not the inorder listing of the nodes
- Insertion and deletion require log(n, 2) operations.

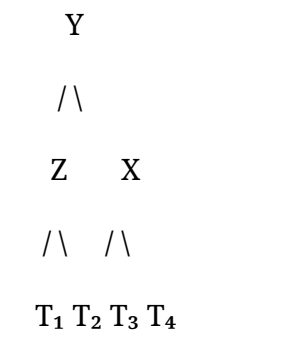
Generalized way of expressing the rotations:

The most useful summary

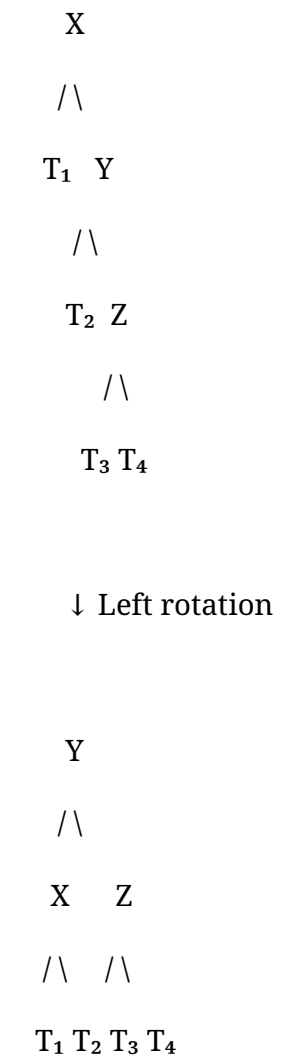
LL



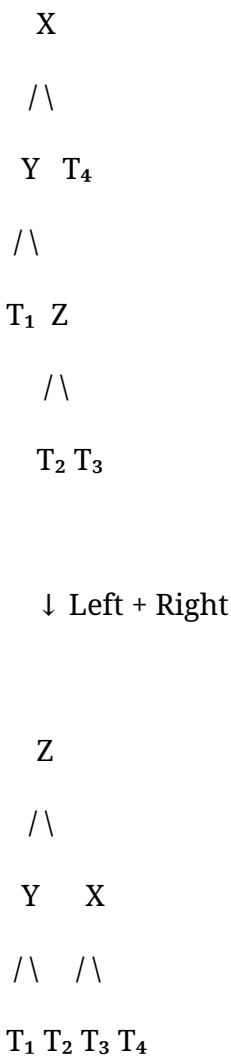
↓ Right rotation



RR

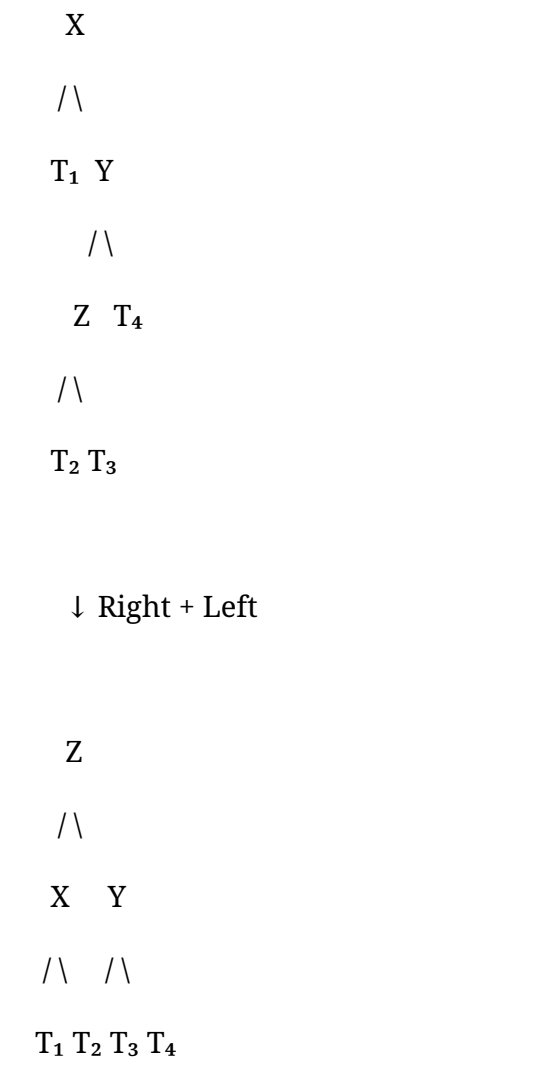


LR



↓ Left + Right

RL



There is another similar tree called the RED BLACK tree which is most commonly used in the libraries.

Amazing idea. Is it not?

Splay Trees

A few points:

A **Splay Tree** is a **self-adjusting Binary Search Tree (BST)**.

Whenever a node is accessed, move that node to the root.

This operation is called **splaying**.

There is no concept of balance factor.

Features:

- **No Extra Metadata:** They require **no balance accounting data** (like color bits in red-black trees or height numbers in AVL trees), saving memory.
- **Locality of Reference:** If we access a piece of data once, it moves to the top, making **subsequent lookups of the same data instantaneous**.
- **Efficiency:** While a single worst-case operation can take time proportional to the number of elements, a sequence of m operations is guaranteed to run in time proportional to m * log(n, 2).
- **Dynamic Adaptation:** The tree structure continuously morphs to fit the **actual access pattern** of the workload rather than enforcing a rigid shape.

Uses:

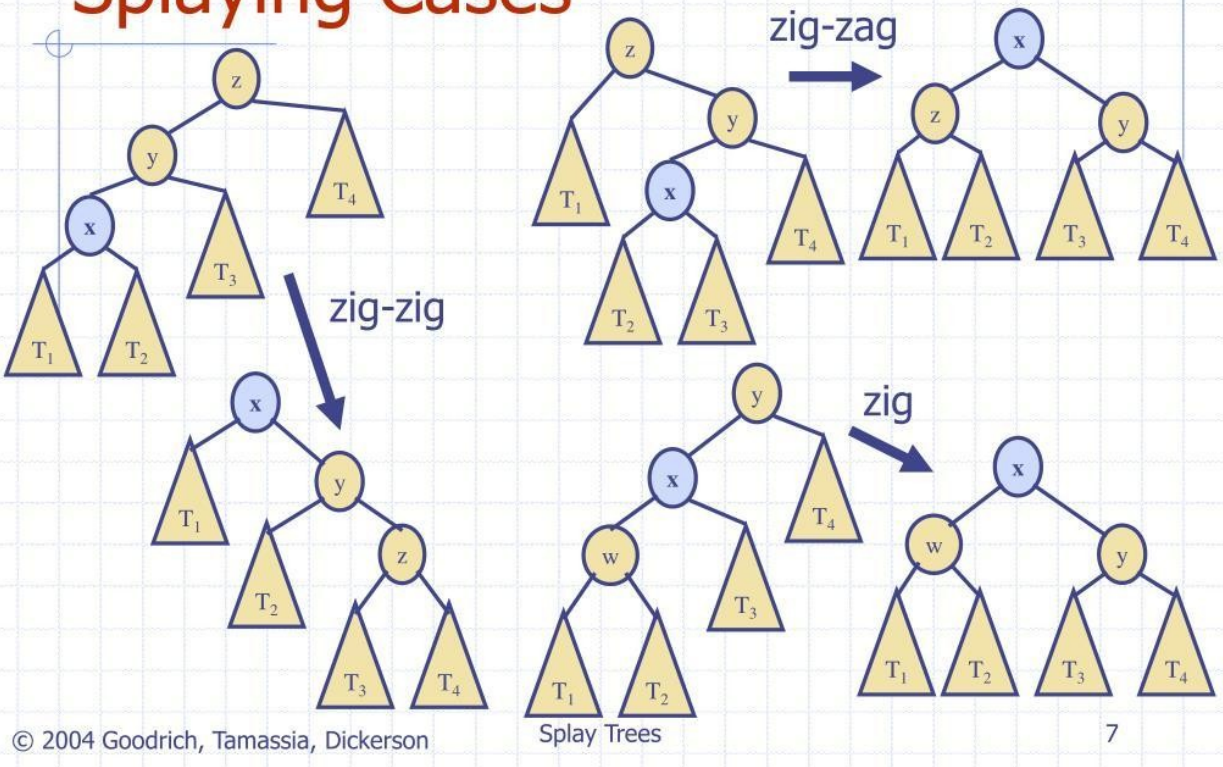
- **Caches and Memory Allocation:** Great for systems where a small subset of data is requested repeatedly.
- **Network Routers:** Used to quickly look up frequently matched IP addresses.
- **Data Compression:** Ideal for algorithms that exploit recurring patterns in data streams.

The first diagram in the next page illustrate the three basic operations : Zig-Zig, Zig-zag and Zig. We shall discuss them in detail.

The second diagram illustrates what happens when the key 2 is searched. This key will move to the root.

Frequently accessed elements move to the root. The principle of the splat tree is that the key accessed recently is likely to be accessed again in the near future.

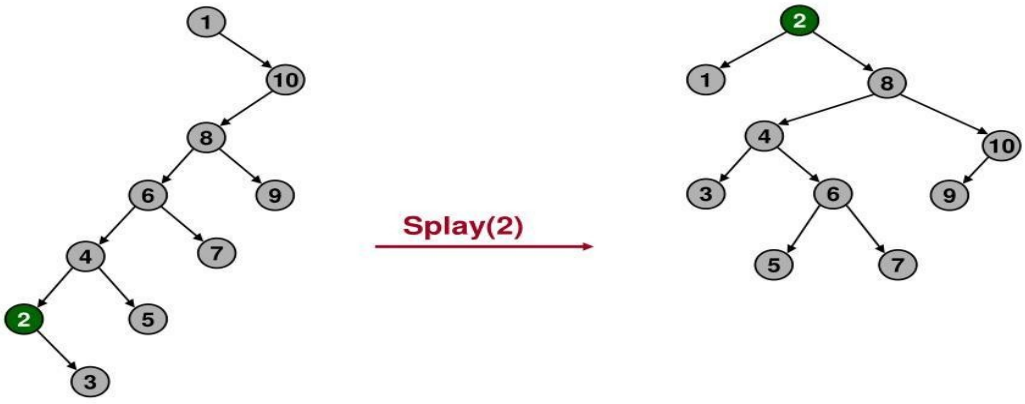
Visualizing the Splaying Cases



© 2004 Goodrich, Tamassia, Dickerson

Splay Example

Apply Splay(2, S) to tree S:



How is a node moved to the root?

Splaying is performed using **rotations**.

There are three basic cases:

1. **Zig**
2. **Zig-Zig**
3. **Zig-Zag**

1. Case 1 : Zig:

   This occurs when the node being accessed is a **child of the root**.

   Suppose we access 30:

```
    50
   /
  30
```

   Perform a **right rotation**:

```
   30
    \
     50
```

   So 30 becomes the root.

   Suppose we access 70

```
   50
    \
     70
```

   perform a left rotation:

```
    70
   /
```

50

Zig rule

Node is child of root

↓

Perform one rotation

↓

Node becomes root

Case 2 — Zig-Zig

This occurs when the node and its parent are both on the **same side**.

For example:

```
      50
     /
    30
   /
  20
```

We want to access 20.

Here:

- 20 is the **left child** of 30
- 30 is the **left child** of 50

So this is **Left-Left (LL)**.

First rotation

Rotate 50 right:

```
      30
     /  \
```

20 50

Second rotation

Rotate 30 right:

```
    20
     \
      30
       \
        50
```

So:

```
    50              20
   /                  \
  30      →           30
 /                      \
20                      50
```

This is **Zig-Zig**.

---

5. Another Zig-Zig: Right-Right

Consider:

```
  20
   \
    30
     \
```

50

Access 50.

Both links are to the **right**.

Therefore, it is a **Right-Right (RR)** case.

After two left rotations:

50

/

30

/

20

So:

LL → two right rotations

RR → two left rotations

Case 3 — Zig-Zag

Here the node and its parent are on **opposite sides**.

Consider:

50

/

20

```
    \
     30
```

We want to access 30.

Here:

- 30 is the **right child** of 20
- 20 is the **left child** of 50

Therefore:

Left-Right (LR) configuration.

First rotation

Rotate 20 left:

```
      50
     /
    30
   /
  20
```

Second rotation

Rotate 50 right:

```
     30
    /  \
   20   50
```

So 30 becomes the root.

The four rotation patterns

Configuration	**Splay operation**
Node is child of root	**Zig**

Configuration	Splay operation
Left-Left	**Zig-Zig**
Right-Right	**Zig-Zig**
Left-Right	**Zig-Zag**
Right-Left	**Zig-Zag**

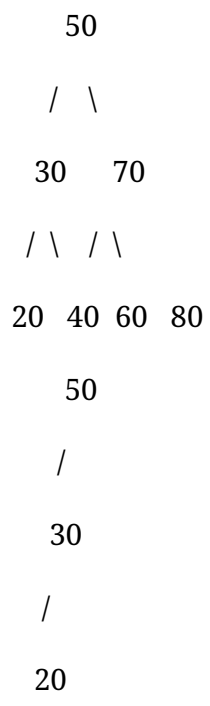
A useful way to remember it:

Same direction → Zig-Zig

Opposite direction → Zig-Zag

One level from root → Zig

Example 1: search for 20



The path is:

50 → 30 → 20

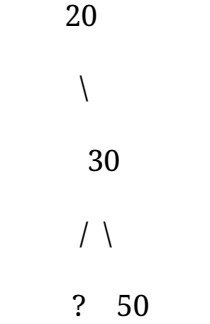
Since:

20 is left child of 30

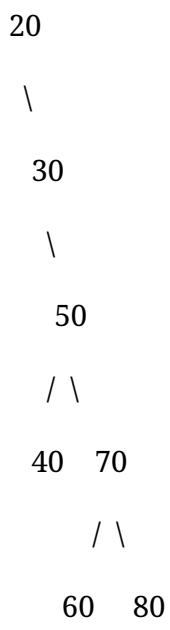
30 is left child of 50

we have **Zig-Zig**.

After splaying:



Including the remaining nodes, the resulting tree is:



The important point is not that the tree remains perfectly balanced.

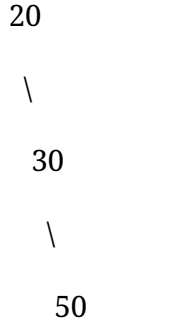
The important point is:

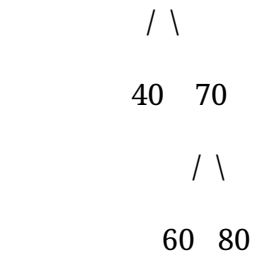
20, which was just accessed, is now at the root.

Example 2:

What happens if we now access 60?

Current tree:





The path to 60 is:

20 → 30 → 50 → 70 → 60

The node 60 is:

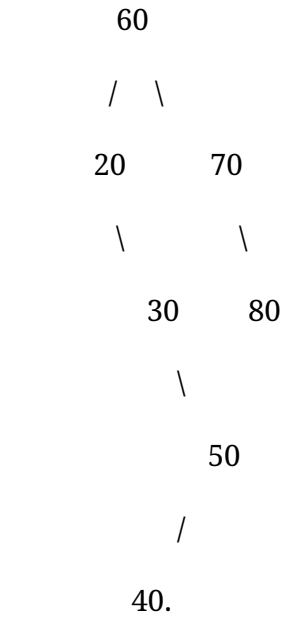
60 is left child of 70

70 is right child of 50

This is a **Zig-Zag** configuration.

After appropriate rotations, 60 moves upward.

Eventually:



and 60 becomes the root.

Thus, every access modifies the tree.

Graph

A few points:

So far, we have discussed linear data structures like list, stack, queue, circular list and doubly linked list.

We have also studied non linear structure tree and its variations.

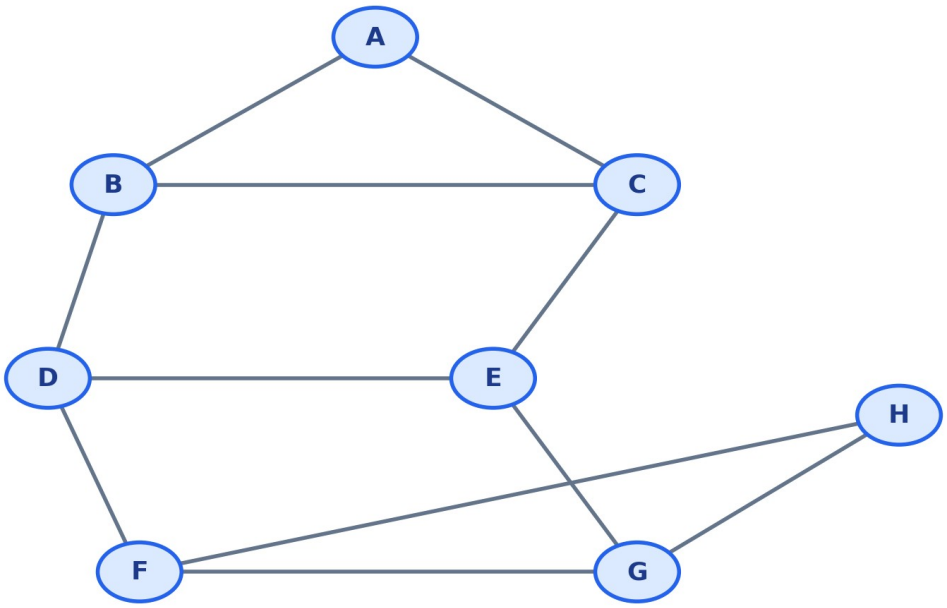
We shall discuss a new data structure called the graph

A **graph** is a **non-linear data structure** used to represent relationships between objects. It consists of vertices (nodes) and edges connecting pairs of vertices.

G = (V, E)

V is the set of vertices and E is the set of edges.

Let us take an example.



There are 8 vertices or nodes.

V = { A, B, C, D, E, F, G, H}

There are 11 edges.

AB, AC, BC, BD, CE, DE, DF, FG, EG, GH, FH

A graph may be directed or non-directed. The above graph is not directed.

Each edge may have some value associated with it – called weight. Such a graph is a weighted graph.

It could be the distance between a pair of geographical locations represented by the vertices, time or cost, …

Terminolgies :

- **Vertex (node):** An object represented in the graph, such as A.
- **Edge:** A connection between two vertices, such as AB.
- **Adjacent vertices:** Vertices connected directly by an edge; A and B are adjacent.
- **Incident edge:** An edge is incident on each endpoint; AB is incident on A and B.
- **Degree:** The number of edges incident on a vertex in an undirected graph.
- **In-degree:** The number of directed edges entering a vertex.
- **Out-degree:** The number of directed edges leaving a vertex.
- **Path:** A sequence of vertices joined by edges, such as A–B–D–F–H.
- **Path length:** The number of edges in a path; A–B–D–F–H has length 4.
- **Cycle:** A path returning to its starting vertex without repeating intermediate vertices, such as A–B–C–A.
- **Connected graph:** An undirected graph where every vertex is reachable from every other vertex.
- **Connected component:** A maximal connected part of a graph.
- **Simple graph:** A graph without self-loops or multiple edges between the same pair of vertices.
- **Self-loop:** An edge connecting a vertex to itself.
- **Complete graph:** A simple undirected graph where every pair of distinct vertices is connected.
- **Subgraph:** A graph formed from a subset of another graph's vertices and edges.
- **Cyclic / acyclic graph:** A graph that contains / does not contain a cycle.

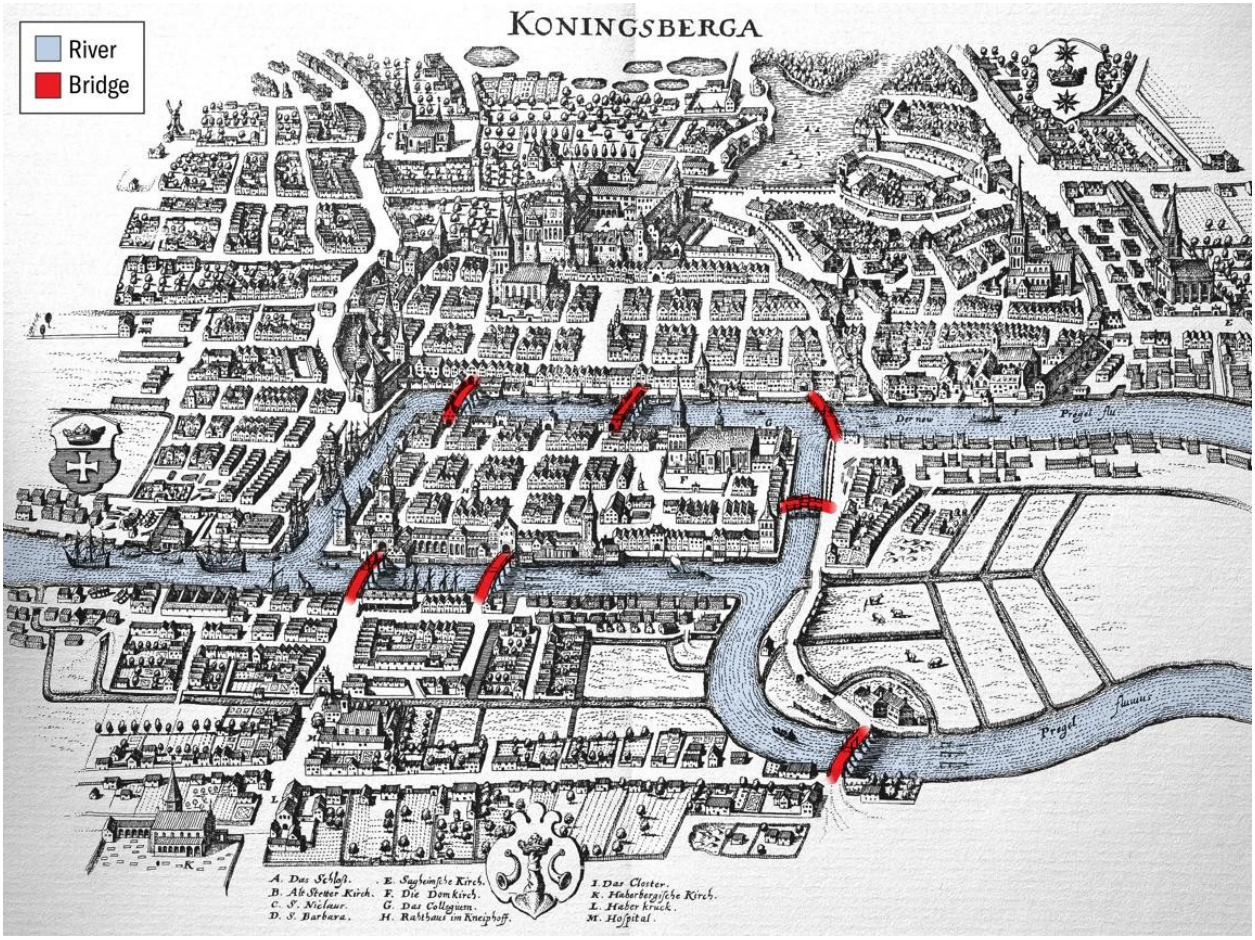
In this graph, we observe the following.

- Number of vertices: $|V| = 8$.
- Number of edges: $|E| = 11$.
- Neighbours of A: B and C.
- Degree of A: $\deg(A) = 2$.
- Degree of B: $\deg(B) = 3$ (edges AB, BC and BD).
- One path from A to H is A–B–D–F–H; its length is 4.
- A cycle is A–B–C–A.

- The graph is connected because every vertex can be reached from every other vertex.

For an undirected graph, $\Sigma \deg(v) = 2|E|$ because every edge contributes one degree to each endpoint. Here the degree sum is $22 = 2 \times 11$.

Königsberg Bridge Problem, Euler Graphs and Hamiltonian Graphs



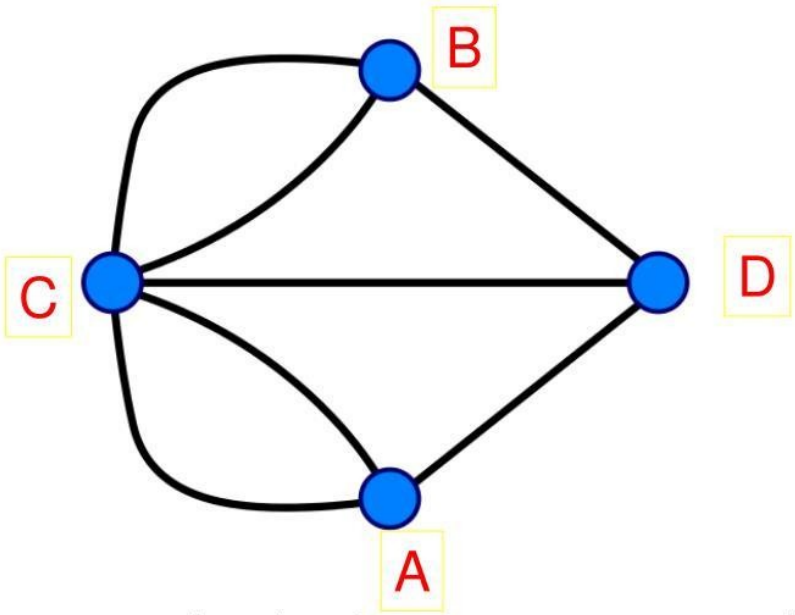
Königsberg Bridge Problem is a classical problem in graph theory.

There are 7 bridges across the rivers. The challenge is to start from any point, traverse the 7 bridges once and only once and return to the starting point. We cannot take any other means of travel.

This problem is supposed to have remained unsolved for a long time until Euler proved that it is not possible.

The simplified bridge map may be shown as in the next page. The requirement is draw the diagram without lifting the pen.

The seven bridges of Konigsberg =



Can you start and end at the same vertex, traversing every edge only once?

Euler's solution is simple. If we start from a vertex and end there, we require two edges. We enter vertex and leave it each time we require two edges. So, only if the degree of each vertex is even, it is possible to traverse a graph without repeating an edge.

A graph is said to be Eulerian if it supports the traversal of edges once and only once and come back to the starting point. The necessary and sufficient condition is that the degree of each node should be even.

What if want to visit each vertex only once and come back to the starting point? This is called Hamiltonian cycle. Even though this appears similar to Euler cycle, till today we do not have a simple solution to determine whether a Hamiltonian circuit exists for a given graph.

Applications of graphs:

- **Road and transport networks:** Vertices are cities or intersections; edges are roads. Weights can represent distance or travel time.
- **Social networks:** Vertices are people; edges are friendships, follows or interactions.
- **Computer networks:** Vertices are computers or routers; edges are communication links.
- **Course prerequisites:** A directed edge can mean one course must be completed before another.
- **Software dependencies:** Vertices are modules, files or tasks; edges represent dependencies.
- **Airline and logistics networks:** Vertices are airports or warehouses; edges represent routes.
- **Web pages:** Vertices are pages and directed edges represent hyperlinks.
- **Project scheduling:** Vertices represent tasks and edges represent precedence constraints

Abstract Data Type:

Operation	Purpose
create_graph()	Create an empty graph.
add_vertex(v)	Insert a vertex.
remove_vertex(v)	Remove a vertex and its incident edges.
add_edge(u, v)	Add an edge between u and v.
remove_edge(u, v)	Remove an edge.
has_vertex(v)	Check whether v exists.
has_edge(u, v)	Check whether an edge exists.
get_neighbors(v)	Return vertices adjacent to v.
degree(v)	Return the degree of v in an undirected graph.
display_graph()	Display vertices and edges.
destroy_graph()	Release resources used by the graph.

If the graph is directed, we should consider the direction. Adding an edge u to v will not be same as adding an edge v to u.

If the graph is weighted, we may want to set and/or get weights from the edges.

Graph representation

A. Adjacency matrix

An adjacency matrix A is a V × V array. For an unweighted graph, A[i][j] is 1 when an edge exists and 0 otherwise. For an undirected graph, the matrix is symmetric. The 8-vertex example requires an 8 × 8 matrix.

Space: $O(V^2)$. Checking whether an edge exists takes O(1) time.

The space required is proportional to the square of the number of vertices.

The time to check whether there is an edge between the vertices i and j requires constant time – does not depend on the number of vertices or number of edges.

B. Adjacency list

An adjacency list stores a list of neighbours for each vertex. For the example graph:

A → B, C
B → A, C, D
C → A, B, E
D → B, E, F
E → C, D, G
F → D, G, H
G → E, F, H
H → F, G
Space required will be a node for each vertex and another node for each edge.

There are other ways of representing the graph – we shall not discuss in this course.